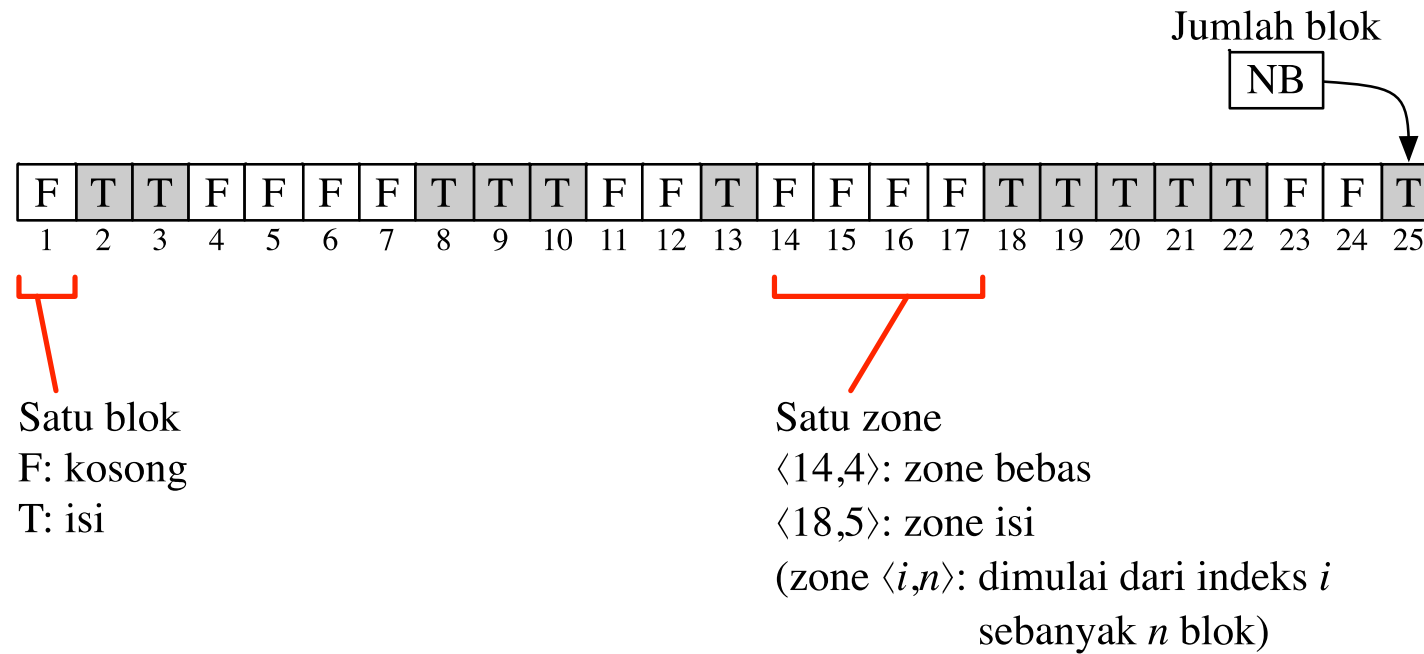


Studi Kasus: Pengelolaan Memori Rep. Berkait – Blok Kosong

IF2110/IF2111 – Algoritma dan Struktur Data
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

Memori

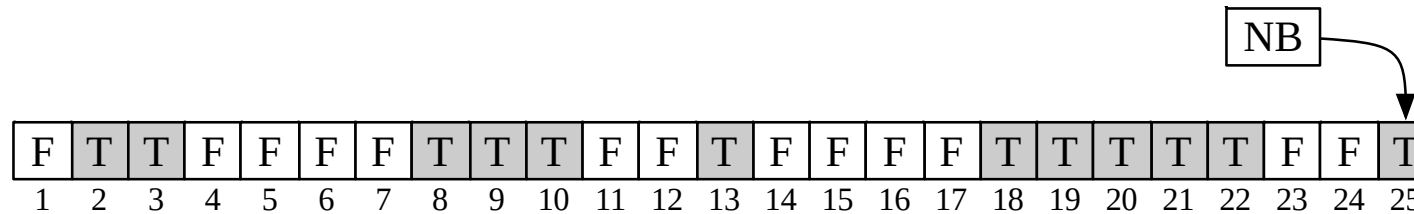


Deskripsi Persoalan

Memori dinyatakan sebagai NB buah blok kontigu

F: KOSONG

T: ISI



Zone Bebas: blok-blok berurutan yang berstatus F (KOSONG)

Dinyatakan oleh indeks blok KOSONG pertama dan ukurannya

Zone bebas I: $\langle 1,1 \rangle$

Zone bebas II: $\langle 4,4 \rangle$

Zone bebas III: $\langle 11,2 \rangle$

Zone bebas IV: $\langle 14,4 \rangle$

Zone bebas V: $\langle 23,2 \rangle$

Deskripsi Persoalan

Alokasi dan dealokasi memori menyebabkan perubahan terhadap zone bebas
Realisasikan prosedur-prosedur sebagai berikut:

Prosedur InitMem

Mengeset semua blok menjadi blok KOSONG

Prosedur AlokBlok

Melakukan alokasi: membuat blok KOSONG sejumlah X menjadi ISI

Menghasilkan indeks awal (IAw) di mana alokasi dilakukan

Prosedur DeAlokBlok

“Membebaskan” zone ISI: membuat blok ISI sejumlah X yang berawal di indeks IAw menjadi KOSONG

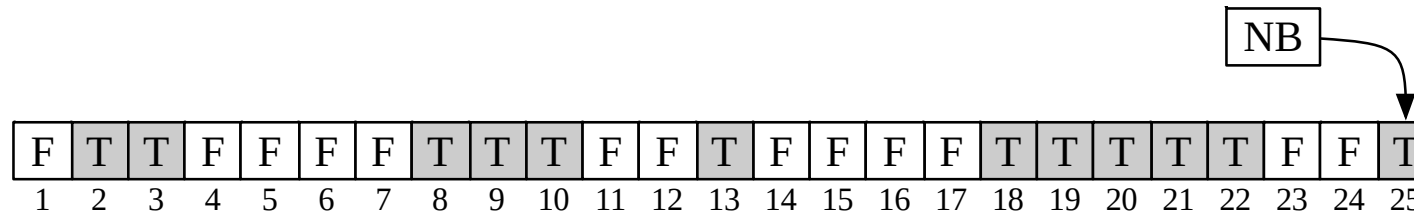
Prosedur GarbageCollection

Memampatkan memori sehingga semua blok KOSONG berada di bagian kiri memori dan semua blok ISI berada di kanan memori

Representasi Berkait Blok Kosong

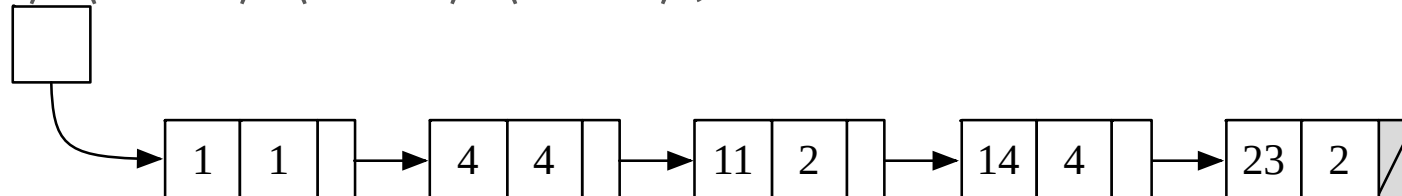
Representasi Berkait Blok Kosong

Ilustrasi



List zone kosong:

{ $\langle 1,1 \rangle$, $\langle 4,4 \rangle$, $\langle 11,2 \rangle$, $\langle 14,4 \rangle$, $\langle 23,2 \rangle$ }



Representasi Berkait Blok Kosong

Struktur Data

KAMUS

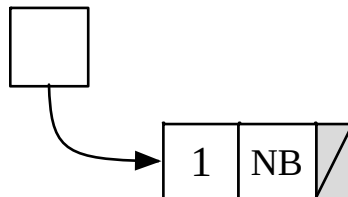
```
type: address          { type terdefinisi }
type ZB: < Aw: integer, { indeks awal zone kontigu kosong }
           Nb: integer, { banyaknya blok zone kontigu kosong }
           Next: address >
FIRSTZB: address        { address zone kosong pertama }
{ Elemen list terurut menurut Aw }
{ Fungsi akses untuk penulisan algoritma secara logik }
{ Jika P adalah sebuah address, maka dituliskan:
  - Next(P) adalah address elemen list sesudah elemen beralamat P
  - Aw(P) adalah nilai blok kosong pertama pada sebuah elemen
    list beralamat P yang mewakili sebuah zone kontigu
  - Nb(P) adalah ukuran blok sebuah zone kosong yang disimpan
    informasinya dalam elemen list beralamat P
  - Allocate(P) adalah prosedur utk melakukan alokasi sebuah ZB
    dengan alamat P, P tidak mungkin sama dengan Nil (alokasi
    selalu berhasil)
  - Deallocate(P) adalah prosedur utk mendealokasi sebuah alamat P }
```

Representasi Berkait Blok Kosong

Prosedur InitMem

Algoritma: diktat hlm. 176

```
procedure InitMemB
{ I.S.: Sembarang }
{ F.S.: Semua blok memori dinyatakan KOSONG }
{ Proses: Diinisialisasi satu zone kosong <1, NB>
        jika NB adalah jumlah maksimum blok. }
{ Solusi umum:
  1. Create list kosong
  2. Allocate sebuah elemen baru P dengan NAw=1 dan Ukuran NB
  3. Insert elemen P dalam List }
```



Representasi Berkait Blok Kosong

Prosedur AlokBlok – First Fit (1)

```
procedure AlokBlokBF (input X: integer, output IAw: integer)
{ I.S.: Sembarang. X adalah banyaknya blok yang diminta untuk
  dialokasi, yaitu dijadikan ISI }
{ F.S.: Tergantung kepada proses }
{ Proses: Sequential search sebuah elemen list dengan properti
  jumlah blok kontigunya lebih besar atau sama dengan X.
  Pencarian segera dihentikan jika diketemukan elemen
  list yang memenuhi persyaratan tersebut. }
{ Hasil pencarian menentukan F.S.:
  1. Jika ada, maka ada dua kemungkinan:
    a. jika jumlah blok kontigu sama dengan X, hapus elemen
       list kosong tersebut,
    b. jika jumlah blok kontigu lebih besar dari X,
       update elemen list kosong tersebut.
  2. Jika tidak ada elemen list yang memenuhi syarat:
     keadaan list tetap dan IAw diberi nilai 0. }
```

Representasi Berkait Blok Kosong

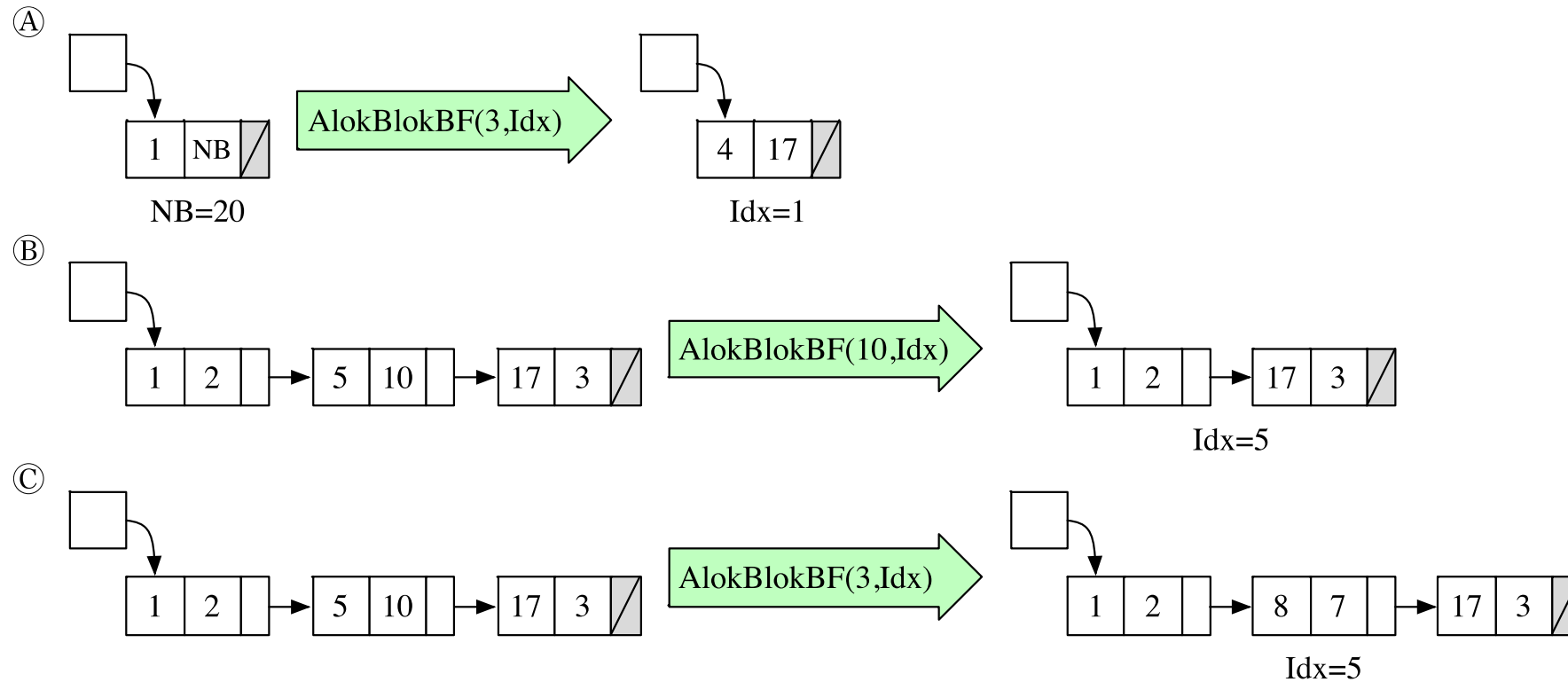
Prosedur AlokBlok – First Fit (2)

Algoritma: diktat hlm. 177

Sketsa umum algoritma:

```
Sequential search List FirstZB, P sebuah address elemen
kondisi berhenti semua elemen list diperiksa atau  $Nb(P) \geq X$ 
if ( $Nb(P) \geq X$  then      { ada yg memenuhi syarat }
    if  $Nb(P) = X$  then { zone kosong menjadi isi }
        Delete elemen beralamat P; Dealokasi P
    else { lebih besar: update zone kosong }
        Update  $Nb(P)$  dan  $Aw(P)$ 
        Set nilai  $IAw$  dengan  $Aw(P)$ 
else Set  $IAw = 0$ 
```

Ilustrasi Alokasi: First Fit



Representasi Berkait Blok Kosong

Prosedur AlokBlok – Best Fit (1)

procedure AlokBlokBB (input X: integer, output IAw: integer)

{ I.S.: Sembarang.

 X adalah banyaknya blok yang diminta untuk dialokasi,
 yaitu status memorinya dijadikan ISI }

{ F.S.: Tergantung kepada proses }

{ Proses: Periksa semua elemen list, elemen list dengan properti
 jumlah blok kontigunya lebih besar atau sama dengan X
 ditandai yang minimum. }

{ Setelah semua elemen diperiksa, ada dua kemungkinan:

1. Jika alokasi dapat dilakukan, ada blok yang memenuhi syarat,
 masih ada dua kemungkinan:
 - a. jika jumlah blok kontigu sama dengan X, hapus elemen list
 kosong tersebut,
 - b. jika jumlah blok kontigu lebih besar dari X, update elemen
 list kosong tersebut.
2. Jika alokasi tidak dapat dilakukan (tidak ada blok yang
 memenuhi syarat), maka list tetap keadaannya dan IAw diberi
 nilai 0 }

Representasi Berkait Blok Kosong

Prosedur AlokBlok – Best Fit (2)

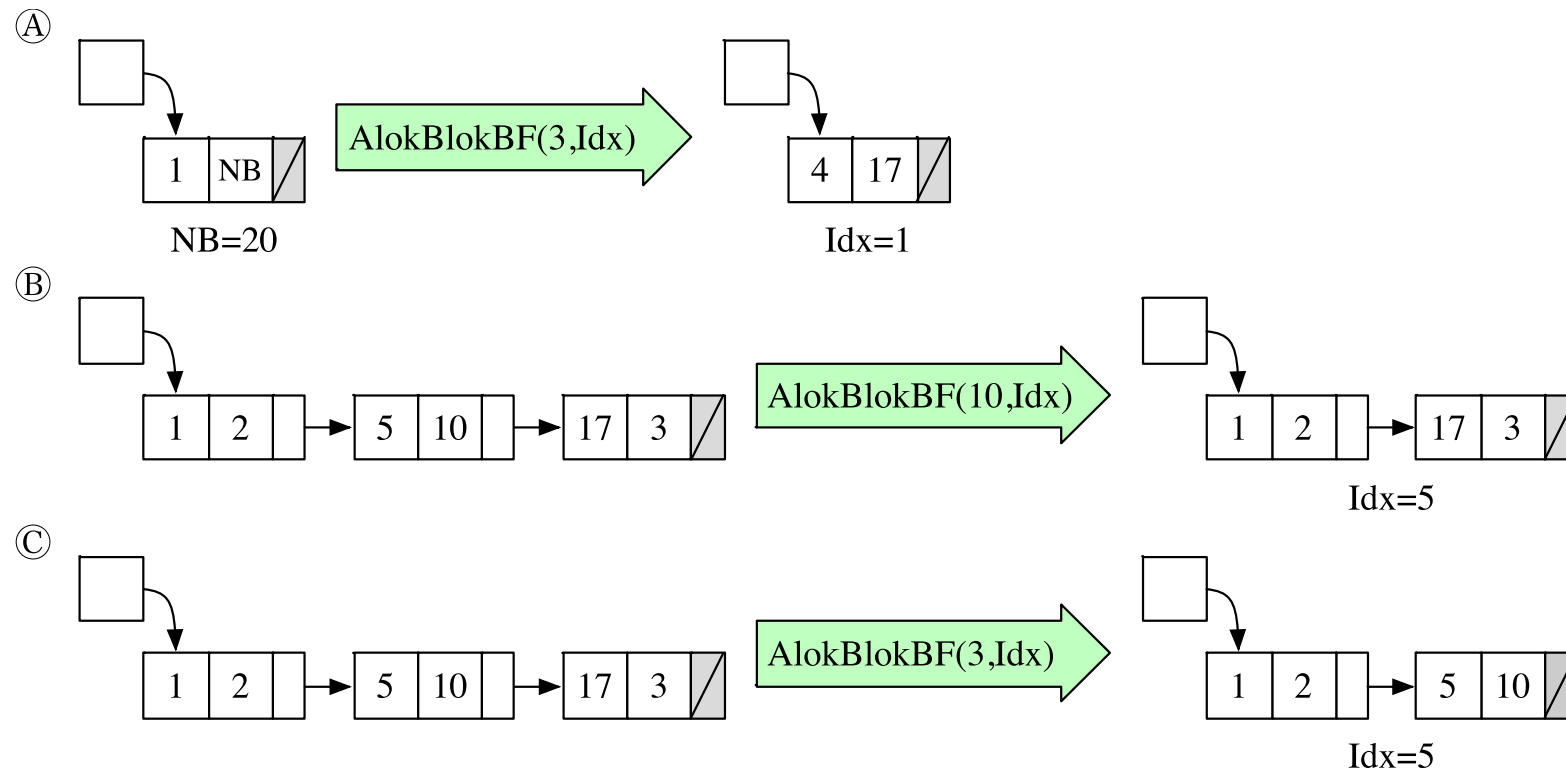
Algoritma: diktat hlm. 178

Sequential search List FirstZB, P sebuah address elemen
kondisi berhenti $Nb(P) = X$ atau semua elemen list diperiksa
Skema search dengan boolean.

Untuk setiap elemen list beralamat P yang diperiksa:

```
if elemen pertama then  
    Inisialisasi NBMin dan Naw  
else { bukan elemen pertama }  
    Cek apakah  $Nb(P) < NBMIN$ , jika ya update NBMin}  
if ( $Nb(P) \geq X$ ) then { ada yg memenuhi syarat }  
    if ( $Nb(P) = X$ ) then { zone kosong menjadi isi }  
        Delete elemen beralamat P; dealokasi P  
    else { lebih besar: update zone kosong }  
        Update  $Nb(P)$  dan  $Aw(P)$   
        Set nilai IAw dengan  $Aw(P)$   
else Set IAw = 0
```

Ilustrasi Alokasi: Best Fit



Representasi Berkait Blok Kosong

Prosedur DeAlokBlok (1)

Algoritma: diktat hlm. 179

```
procedure DeAlokBlokB (input IAw: integer, input X: integer)  
{ I.S.: X adalah ukuran zone, bilangan positif dan IAw adalah  
      alamat blok awal zone tersebut, dengan  $IAw \in [1..NB-X]$ ,  
      blok dengan indeks IAw s.d.  $IAw+X-1$  pasti berstatus ISI. }  
{ F.S.: Tabel status memori dengan indeks blok  $IAw..IAw+X-1$   
      menjadi KOSONG. }  
{ Proses: Sebuah zone berukuran X dan berawal pada blok IAw  
      didealokasi (statusnya dijadikan KOSONG). }
```

Representasi Berkait Blok Kosong

Prosedur DeAlokBlok (2)

Kasus-kasus pada proses dealokasi:

- Zone yang dibebaskan mengubah elemen pertama list:

 - Hanya mengubah elemen pertama list

 - Insert first → menambah zone bebas di awal list

- Zone yang dibebaskan mengubah elemen terakhir list:

 - Hanya mengubah elemen terakhir list

 - Insert last → menambah zone bebas di akhir list

- Zone yang dibebaskan berada di tengah list, di antara elemen KIRI dan KANAN:

 - KIRI dan KANAN digabung → salah satu di-delete, lainnya di-update

 - Di tengah KIRI dan KANAN, tapi tidak bersambung → insert elemen baru di antara KIRI dan KANAN

 - Terletak sesudah elemen KIRI → update KIRI

 - Terletak sebelum elemen KANAN → update KANAN

Jauh lebih rumit daripada representasi secara kontigu!

Representasi Berkait Blok Kosong

Prosedur GarbageCollection

Algoritma: Diktat hlm. 180

```
procedure GarbageCollectionB
{ I.S.: Sembarang }
{ F.S.: Semua blok kosong ada di kiri dan blok isi di kanan:
    Jika ada zone kosong, hanya ada satu elemen list,
    karena dijadikan satu zone kosong.
    Jika tidak list zone kosong tidak ada elemennya (kosong),
    maka tidak dilakukan apa-apa. }
{ Proses: Jika list zone kosong tidak kosong, jadikan sebuah
    list dengan elemen tunggal beralamat P,
    Aw(P) bernilai 1 dan Nb(P) bernilai total blok kosong
    (dihitung melalui iterasi list zone kosong yang lama). }
```